GuardianBand — ESP32 Fall-Detection & SOS Wearable — PRD

Objective

Create firmware for an ESP32 DevKit worn by an elderly or lone worker. The device continuously reads a 3-axis accelerometer, detects a fall on-device, gives the wearer a short window to cancel a false alarm, raises a local alarm (buzzer + LED), and broadcasts its live safety state over BLE advertising so a nearby caregiver phone or laptop can see it without pairing.

Problem

Falls at home or on a work site often go unnoticed for hours. Camera systems raise privacy concerns and need infrastructure. A low-cost wearable that detects the fall itself and broadcasts an alert needs no camera, no Wi-Fi and no app pairing.

Hardware

Item	Connection
ESP32 DevKit (ESP32-WROOM-32)	USB to laptop (power + UART)
MPU6050 accelerometer/gyro module (I2C address 0x68)	SDA = GPIO21, SCL = GPIO22, VCC = 3V3, GND = GND
Active buzzer (3.3 V, digital on/off)	GPIO25 to buzzer +, buzzer − to GND
Alert LED with 220 Ω resistor	GPIO26 to resistor to LED anode, LED cathode to GND
Push button	On-board BOOT button (GPIO0, active low, use internal pull-up)

No other hardware is required.

Device States

The firmware shall implement exactly these four safety states:

Code	State	Meaning
0	NORMAL	No event. LED off, buzzer off.
1	FALL_SUSPECTED	A fall pattern was detected; the wearer can cancel.
2	FALL_CONFIRMED	Not cancelled in time; help is needed.
3	SOS	The wearer manually requested help.

Requirements

Sensing

1. Initialize the MPU6050 over I2C at 400 kHz after boot, with the accelerometer full-scale range set to ±8 g.

2. If the MPU6050 is not detected at boot, print an error on UART, blink the LED 3 times, and retry initialization every 2 seconds without rebooting.

3. Sample the accelerometer at 100 Hz (every 10 ms) in a dedicated FreeRTOS task.

4. Compute the acceleration magnitude $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$ in g for every sample.

Fall Detection (all thresholds defined as named constants in one configuration header)

5. Free-fall phase: detect $|a| < 0.40$ g sustained for at least 60 ms.

6. Impact phase: after a free-fall phase, detect $|a| > 2.50$ g within the next 500 ms.

7. Stillness phase: after an impact, the device is still when |a| stays between 0.80 g and 1.20 g for 2 seconds (short spikes up to 100 ms are tolerated).

8. When free-fall, impact and stillness occur in that order, change state from NORMAL to FALL_SUSPECTED.

9. Record the peak |a| of the impact phase for broadcasting.

Alert Handling

10. In FALL_SUSPECTED: beep the buzzer 200 ms on / 800 ms off and blink the LED at 1 Hz for a 10-second cancel window.

11. A short press (< 1 s) of the button during the cancel window returns the device to NORMAL and logs "FALL CANCELLED BY USER".

12. If the cancel window expires, change to FALL_CONFIRMED: buzzer on continuously in a 500 ms on / 500 ms off pattern and LED blinking at 4 Hz.

13. In NORMAL, a long press (≥ 2 s) of the button changes the state to SOS with the same buzzer and LED pattern as FALL_CONFIRMED.

14. In FALL_CONFIRMED or SOS, a long press (≥ 3 s) of the button acknowledges the alert and returns to NORMAL.

15. Debounce the button for 50 ms.

16. Each transition into FALL_SUSPECTED, FALL_CONFIRMED or SOS increments an 8-bit event counter (wraps at 255).

BLE Broadcast

17. Initialize BLE after boot and start non-connectable BLE advertising automatically. No GATT services, connections or pairing.

18. Read the ESP32 Bluetooth MAC address at runtime and name the device GuardianBand_XX, where XX is the last 2 uppercase hexadecimal digits of that MAC address.

19. Include the complete local name and Manufacturer Specific Data in the advertising payload. Manufacturer Specific Data layout (8 bytes, little-endian):

Byte	Field	Value
0–1	Company ID	0xFFFF (test ID), sent as FF FF
2	Protocol version	0x01
3	State code	0 NORMAL, 1 FALL_SUSPECTED, 2 FALL_CONFIRMED, 3 SOS
4	Event counter	0–255
5	Last impact peak	peak g × 10, capped at 255 (e.g. 3.4 g → 34)
6–7	Uptime	seconds since boot ÷ 10, uint16 little-endian

20. Update the advertising payload within 500 ms of every state change and at least once every 10 seconds otherwise (uptime refresh).

21. Advertising interval 100 ms in FALL_SUSPECTED, FALL_CONFIRMED and SOS; 500 ms in NORMAL.

22. Keep BLE advertising continuously for the whole time the device is powered.

UART Output (115200 baud)

23. At boot print: firmware name and version "GuardianBand v1.0", the Bluetooth MAC address, the generated BLE name, the MPU6050 init result, and "BLE ADVERTISING STARTED".

24. On every state change print one line: `STATE <old> -> <new> | events=<n> | peak=<g> g`.

25. Once per second while in NORMAL print `ACC |a|=<g> g` (one line, 2 decimals) for live monitoring.

26. Test hook: when the character `f` is received on UART, inject a simulated fall (as if free-fall, impact of 3.0 g and stillness occurred) and log "SIMULATED FALL INJECTED". When `s` is received, behave as a long press in NORMAL (enter SOS).

Architecture

27. Keep sensor driver, fall-detection logic, alert output (buzzer/LED), button handling and BLE advertising in separate modules. The fall-detection logic must be pure C with no hardware calls so it can be unit-tested on the host by feeding it sample sequences.

28. Use ESP-IDF v5.5 with the NimBLE host stack.

Out of Scope

GATT services, BLE connections or pairing, Wi-Fi, cloud, OLED display, step counting, battery

measurement, deep sleep, OTA updates.

Acceptance Criteria

1. The ESP32 boots, prints the boot banner with the MAC address and the GuardianBand_XX name, and prints "BLE ADVERTISING STARTED" with no reboot loop.

2. A BLE scanner phone app shows GuardianBand_XX, and XX matches the last 2 digits of the Bluetooth MAC printed on UART.

3. With the device at rest, UART prints `ACC |a|` = values between 0.90 and 1.10 g once per second.

4. Dropping the device about 50 cm onto a cushion, then leaving it still, changes the state to FALL_SUSPECTED within 3 seconds of landing; the buzzer beeps and the LED blinks.

5. Short-pressing BOOT within 10 seconds returns to NORMAL and prints "FALL CANCELLED BY USER".

6. Not pressing anything for 10 seconds changes the state to FALL_CONFIRMED with the continuous alarm pattern.

7. Long-pressing BOOT for 3 seconds in FALL_CONFIRMED returns to NORMAL.

8. Long-pressing BOOT for 2 seconds in NORMAL enters SOS.

9. Sending `f` over UART triggers FALL_SUSPECTED and sending `s` triggers SOS.

10. The manufacturer data seen by a BLE scanner shows the correct state byte within 1 second of each state change and the event counter increases by one per alert event.

11. Walking, sitting down quickly and shaking the device gently for 30 seconds does not trigger FALL_SUSPECTED.

12. Host unit tests of the fall-detection module pass for: a real fall sequence (detect), free-fall without impact (no detect), impact without stillness (no detect), and normal walking (no detect).